

Python Code

```
1  """
2  Code to calculate the circle that passes through three given points.
3
4  Fill in each function with your code (including fixing the return
5  statement).
6  """
7
8  # imports
9  import math
10 import comp140_module1 as circles
11
12 def distance(point0x, point0y, point1x, point1y):
13     """
14     Computes the distance between two points.
15
16     inputs:
17         -point0x: a float representing the x-coordinate of the first point
18         -point0y: a float representing the y-coordinate of the first point
19         -point1x: a float representing the x-coordinate of the second point
20         -point1y: a float representing the y-coordinate of the second point
21
22     returns: a float that is the distance between the two points
23     """
24     #distance formula
25     return math.sqrt((point0x - point1x) ** 2 + (point1y - point0y) ** 2)
26
27 def midpoint(point0x, point0y, point1x, point1y):
28     """
29     Computes the midpoint between two points.
30
31     inputs:
32         -point0x: a float representing the x-coordinate of the first point
33         -point0y: a float representing the y-coordinate of the first point
34         -point1x: a float representing the x-coordinate of the second point
35         -point1y: a float representing the y-coordinate of the second point
36
37     returns: two floats that are the x- and y-coordinates of the midpoint
38     """
39     # midpoint formula
40     return (point0x + point1x)/2, (point0y + point1y)/2
41
42 def slope(point0x, point0y, point1x, point1y):
43     """
44     Computes the slope of the line that connects two given points.
45
46     The x-values of the two points, point0x and point1x, must be different.
47
48     inputs:
49         -point0x: a float representing the x-coordinate of the first point.
50         -point0y: a float representing the y-coordinate of the first point
```

```

51         -point1x: a float representing the x-coordinate of the second point.
52         -point1y: a float representing the y-coordinate of the second point
53
54     returns: a float that is the slope between the points
55     """
56     # slope formula
57     return (point1y - point0y)/(point1x - point0x)
58
59 def perp(lineslope):
60     """
61     Computes the slope of a line perpendicular to a given slope.
62
63     input:
64         -lineslope: a float representing the slope of a line.
65                     Must be non-zero
66
67     returns: a float that is the perpendicular slope
68     """
69
70     return -1/lineslope
71
72
73 def intersect(slope0, point0x, point0y, slope1, point1x, point1y):
74     """
75     Computes the intersection point of two lines.
76
77     The two slopes, slope0 and slope1, must be different.
78
79     inputs:
80         -slope0: a float representing the slope of the first line.
81         -point0x: a float representing the x-coordinate of the first point
82         -point0y: a float representing the y-coordinate of the first point
83         -slope1: a float representing the slope of the second line.
84         -point1x: a float representing the x-coordinate of the second point
85         -point1y: a float representing the y-coordinate of the second point
86
87     returns: two floats that are the x- and y-coordinates of the intersection
88     point
89     """
90     #y = m1 * x - m1 * x1) + y1
91     #y = m2 * x - m2 * x2) + y2
92     # x_intersect = - m1 * x1 + y1 - y2 + m2 * x2 / (m2 - m1)
93     # y_intersect = y = m2 * x - m2 * x2) + y2
94     # point slope form to find coords of intersection
95
96     x_intersect = (point0y - slope0 * point0x - point1y + slope1 * point1x) / (slope1
- slope0)
97     y_intersect = slope1 * x_intersect - slope1 * point1x + point1y
98
99     return x_intersect, y_intersect
100
101 def make_circle(point0x, point0y, point1x, point1y, point2x, point2y):
102     """
103     Computes the center and radius of a circle that passes through

```

```
104     thre given points.
105
106     The points must not be co-linear and no two points can have the
107     same x or y values.
108
109     inputs:
110         -point0x: a float representing the x-coordinate of the first point
111         -point0y: a float representing the y-coordinate of the first point
112         -point1x: a float representing the x-coordinate of the second point
113         -point1y: a float representing the y-coordinate of the second point
114         -point2x: a float representing the x-coordinate of the third point
115         -point2y: a float representing the y-coordinate of the third point
116
117     returns: three floats that are the x- and y-coordinates of the center
118     and the radius
119     """
120     #Connect a pair of points with a line.
121     #Find the midpoint of that line.
122     #Draw a new line that is perpendicular to the original line and passes through the
midpoint.
123     #Repeat steps 1 through 3 for another pair of points.
124     # The point at which the two perpendicular lines intersect is the center of the
circle.
125     # The distance from the center to any of the three original points is the radius
126
127     midpoint1_x, midpoint1_y = midpoint(point0x, point0y, point1x, point1y)
128     midpoint2_x, midpoint2_y = midpoint(point1x, point1y, point2x, point2y)
129     perp_1 = perp(slope(point0x, point0y, point1x, point1y))
130     perp_2 = perp(slope(point1x, point1y, point2x, point2y))
131     intersect_x, intersect_y = intersect(perp_1, midpoint1_x,
132                                         midpoint1_y, perp_2, midpoint2_x, midpoint2_y)
133     radius = distance(intersect_x, intersect_y, point0x, point0y)
134
135     return intersect_x, intersect_y, radius
136
137     # Run GUI - uncomment the line below after you have
138     #         implemented make_circle
139     circles.start(make_circle)
140
```